LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
MODUL 4

Double Link List

Oleh:

Aufa Thariq Akram NIM. 2510817310014

PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
APRIL 2026

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA
MODUL 4

Laporan Praktikum Algoritma & Struktur Data Modul 4: Double Link List ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Prakitkum ini dikerjakan oleh:

Nama Praktikan : Aufa Thariq Akram
NIM : 2510817310014

Menyetujui,	Mengetahui,
Asisten Praktikum	Dosen Penanggung Jawab Praktikum
Fadhil Syahdama Mahatma Putra	Andreyan Rizky Baskara, S.Kom., M.Kom
NIM. 2410817210026	NIP. 199307032019031011

DAFTAR ISI

LEMBAR PENGESAHAN i
DAFTAR ISI ii
DAFTAR GAMBAR iii
DAFTAR TABEL iv
SOAL 1 1
Pembahasan 8
SOAL 2 27
Pembahasan 27
SOAL 3 28
Pembahasan 28
SOAL 4 29
Pembahasan 29
LINK GITHUB 31

DAFTAR GAMBAR

Gambar 1 Screenshot Output Program Soal 1 "Tampilan awal Program" 17
Gambar 2. Screenshot Output Program Soal 1 "Opsi 1" 17
Gambar 3 Screenshot Output Program Soal 1 "Opsi 2" 17
Gambar 4 Screenshot Output Program Soal 1 "Opsi 3" 18
Gambar 5 Screenshot Output Program Soal 4 "Input 1 2 3” 29
Gambar 6 Screenshot Output Program Soal 4 "Hasil input 1 2 3” 30

DAFTAR TABEL

Tabel 1. Source Code Soal Nomor 1 8

SOAL 1

Lengkapi coding pada function tambahDepanH() agar bisa berjalan dengan lancar. running, simpan program, dan screenshoot hasil running !

```
#include <conio.h>
#include <iostream>
#include <stdlib.h>

using namespace std;

typedef struct TNode {
    string data;
    TNode *next;
    TNode *prev;
};

TNode *head, *tail;

int pil, menu;
char pilihan[1];
string dataBaru;

void initH();
void initHT();
int isEmptyH();
int isEmptyHT();

void tambahDepanH();
void tambahDepanHT();
void tambahBelakangH();
void tambahBelakangHT();
void hapusDepanH();
void hapusDepanHT();
void hapusBelakangH();
void hapusBelakangHT();
void tampilkanH();
void tampilkanHT();
void clearH();
void clearHT();

int main()
{
    menu:
    cout<<"Double Linked List Non Circular (DLLNC)"<<endl;
    cout<<"========================================"<<endl;
    cout<<"Silahkan pilih program DLLNC yang ingin dijalankan!"<<endl;
    cout<<"1. DLLNC dengan Head"<<endl;
    cout<<"2. DLLNC dengan Head dan Tail"<<endl;
    cout<<"3. Quit"<<endl;
    cout<<"Pilihan : ";
    cin>>menu;
    system("cls");
```

```
    if(menu==1){
        do {
            cout<<"Double Linked List Non Circular (DLLNC) (Head)"<<endl;
            cout<<"========================================"<<endl;
            cout<<"1. Tambah Depan"<<endl;
            cout<<"2. Tambah Belakang"<<endl;
            cout<<"3. Tampilkan Data"<<endl;
            cout<<"4. Hapus Depan"<<endl;
            cout<<"5. Hapus Belakang"<<endl;
            cout<<"6. Reset"<<endl;
            cout<<"7. Kembali ke Menu"<<endl;
            cout<<"Pilihan : ";
            cin>>pilihan;
            pil=atoi(pilihan);
```

```
63
64          switch(pil) {
65          case 1:
66              tambahDepanH();
67              break;
68          case 2:
69              tambahBelakangH();
70              break;
71          case 3:
72              tampilkanH();
73              break;
74          case 4:
75              hapusDepanH();
76              break;
77          case 5:
78              hapusBelakangH();
79              break;
80          case 6:
81              clearH();
82              break;
83          default:
84              system("cls");
85              goto menu;
86          }
87
88          cout<<"\npress any key to continue"<<endl;
89          getch();
90          system("cls");
91
92      } while (pil<7);
93  } else if(menu==2){
94      do {
```

```
            cout<<"Double Linked List Non Circular (DLLNC) (Head dan Tail)"<<endl;
            cout<<"======================================================"<<endl;
            cout<<"1. Tambah Depan"<<endl;
            cout<<"2. Tambah Belakang"<<endl;
            cout<<"3. Tampilkan Data"<<endl;
            cout<<"4. Hapus Depan"<<endl;
            cout<<"5. Hapus Belakang"<<endl;
            cout<<"6. Reset"<<endl;
            cout<<"7. Kembali ke Menu"<<endl;
            cout<<"Pilihan : ";
            cin>>pilihan;
            pil=atoi(pilihan);

            switch(pil) {
            case 1:
                tambahDepanHT();
                break;
            case 2:
                tambahBelakangHT();
                break;
            case 3:
                tampilkanHT();
                break;
            case 4:
                hapusDepanHT();
                break;
            case 5:
                hapusBelakangHT();
                break;
            case 6:
                clearHT();
                break;
            default:
                system("cls");
                goto menu;
            }

            cout<<"\npress any key to continue"<<endl;
            getch();
            system("cls");

        } while (pil<7);
    } else {
        cout<<"\nTERIMA KASIH"<<endl;
        cout<<"Program was made by Nama (NIM)."<<endl;
    }
}
```

```
143  void initH(){
144      head = NULL;
145  }
146
147  void initHT(){
148      head = NULL;
149      tail = NULL;
150  }
151
152  int isEmptyH(){
153      if(head == NULL) return 1;
154      else return 0;
155  }
156
157  int isEmptyHT(){
158      if(tail == NULL) return 1;
159      else return 0;
160  }
161
162  void tambahDepanH() {
163      cout<<"Masukkan data : ";
164
165
166
167
168
169
170
171
172
173
174
175
176
177      cout << "Data \""<<dataBaru<<"\" berhasil dimasukkan di bagian depan.";
178  }
179
180  void tambahDepanHT() {
181      cout<<"Masukkan data : ";
182      cin>>dataBaru;
183      TNode *baru;
184      baru = new TNode;
185      baru->data = dataBaru;
186      baru->next = NULL;
187      baru->prev = NULL;
188      if(isEmptyHT() == 1) {
189          head = baru;
190          tail = baru;
```

```
    } else {
        baru->next = head;
        head->prev = baru;
        head = baru;
    }
    cout << "Data \""<<dataBaru<<"\" berhasil dimasukkan di bagian depan.";
}

void tambahBelakangH() {
    cout<<"Masukkan data : ";
    cin>>dataBaru;
    TNode *baru, *bantu;
    baru = new TNode;
    baru->data = dataBaru;
    baru->next = NULL;
    baru->prev = NULL;
    if(isEmptyH() == 1) {
        head = baru;
    } else {
        bantu = head;
        while(bantu->next != NULL){
            bantu = bantu->next;
        }
        bantu->next = baru;
        baru->prev = bantu;
    }
    cout << "Data \""<<dataBaru<<"\" berhasil dimasukkan di bagian belakang.";
}

```

```
void tambahBelakangHT() {
    cout<<"Masukkan data : ";
    cin>>dataBaru;
    TNode *baru;
    baru = new TNode;
    baru->data = dataBaru;
    baru->next = NULL;
    baru->prev = NULL;
    if(isEmptyHT() == 1) {
        head = baru;
        tail = baru;
    } else {
        tail->next = baru;
        baru->prev = tail;
        tail = baru;
    }
    cout << "Data \""<<dataBaru<<"\" berhasil dimasukkan di bagian belakang.";
}
```

```
void tampilkanH() {
    TNode *bantu;
    bantu = head;
    if(isEmptyH() == 0) {
        while(bantu != NULL) {
            cout<<bantu->data<<' ';
            bantu = bantu->next;
        }
        cout<<endl;
    } else cout<<"Tidak terdapat data pada Linked List";
}

void tampilkanHT() {
    TNode *bantu;
    bantu = head;
    if(isEmptyHT() == 0) {
        while(bantu != tail->next) {
            cout<<bantu->data<<' ';
            bantu = bantu->next;
        }
        cout<<endl;
    } else cout<<"Tidak terdapat data pada Linked List";
}

void hapusDepanH() {
    TNode *hapus;
    string data;
    if(isEmptyH() == 0) {
        hapus = head;
        data = hapus->data;
        if(head->next != NULL) {
            head = head->next;
            head->prev = NULL;
        } else {
            initH();
        }
        delete hapus;
        cout<<"Data \""<<data<<"\" yang berada di depan telah berhasil dihapus.";
    } else cout<<"Tidak terdapat data pada Linked List";
}

void hapusDepanHT() {
    TNode *hapus;
    string data;
    if(isEmptyHT() == 0) {
        hapus = head;
        data = hapus->data;
```

```
        if(head->next != NULL) {
            head = head->next;
            head->prev = NULL;
        } else {
            initHT();
        }
        delete hapus;
        cout<<"Data \""<<data<<"\" yang berada di depan telah berhasil dihapus.";
    } else cout<<"Tidak terdapat data pada Linked List";
}

void hapusBelakangH() {
    TNode *hapus;
    string data;
    if(isEmptyH() == 0) {
        hapus = head;
        while(hapus->next != NULL){
            hapus = hapus->next;
        }
        data = hapus->data;
        if(head->next != NULL) {
            hapus->prev->next = NULL;
        } else {
            initH();
        }
        delete hapus;
        cout<<"Data \""<<data<<"\" yang berada di belakang telah berhasil dihapus.";
    } else cout<<"Tidak terdapat data pada Linked List";
}

void hapusBelakangHT() {
    TNode *hapus;
    string data;
    if(isEmptyHT() == 0) {
        hapus = tail;
        data = hapus->data;
        if(head->next != NULL) {
            tail = tail->prev;
            tail->next = NULL;
        } else {
            initHT();
        }
        delete hapus;
        cout<<"Data \""<<data<<"\" yang berada di belakang telah berhasil dihapus.";
    } else cout<<"Tidak terdapat data pada Linked List";
}

void clearH() {
```

```
    TNode *bantu, *hapus;
    bantu = head;
    while(bantu != NULL) {
        hapus = bantu;
        bantu = bantu->next;
        delete hapus;
    }
    initH();
    cout<<"Seluruh data pada Linked List telah dibersihkan.";
}

void clearHT() {
    TNode *bantu, *hapus;
    bantu = head;
    while(bantu != NULL) {
        hapus = bantu;
        bantu = bantu->next;
        delete hapus;
    }
    initHT();
    cout<<"Seluruh data pada Linked List telah dibersihkan.";
}
```

Pembahasan

Tabel 1. Source Code Soal Nomor 1

```
#include <iostream>
#include <stdlib.h>
#include <termios.h>
#include <unistd.h>

using namespace std;

int getch() {
    struct termios oldt, newt;
    int ch;
    tcgetattr(STDIN_FILENO, &oldt);
    newt = oldt;
    newt.c_lflag &= ~(ICANON | ECHO);
    tcsetattr(STDIN_FILENO, TCSANOW, &newt);
    ch = getchar();
    tcsetattr(STDIN_FILENO, TCSANOW, &oldt);
    return ch;
}

typedef struct TNode {
    string data;
    TNode *next;
    TNode *prev;
};

TNode *head, *tail;
```

```

int pil, menu;
char pilihan[1];
string dataBaru;

void initH();
void initHT();
int isEmptyH();
int isEmptyHT();

void tambahDepanH();
void tambahDepanHT();
void tambahBelakangH();
void tambahBelakangHT();
void hapusDepanH();
void hapusDepanHT();
void hapusBelakangH();
void hapusBelakangHT();
void tampilkanH();
void tampilkanHT();
void clearH();
void clearHT();

int main()
{
    menu:
    cout<<"Double Linked List Non Circular (DLLNC)"<<endl;

cout<<"======================================"<<endl;
    cout<<"Silahkan pilih program DLLNC yang ingin dijalankan!"<<endl;
    cout<<"1. DLLNC dengan Head"<<endl;
    cout<<"2. DLLNC dengan Head dan Tail"<<endl;
    cout<<"3. Quit"<<endl;
    cout<<"Pilihan : ";
    cin>>menu;
    system("clear");
    if(menu==1){
        do {
            cout<<"Double Linked List Non Circular (DLLNC) (Head)"<<endl;


```

```
68
69  cout<<"==========================================
    ==="<<endl;
70              cout<<"1. Tambah Depan"<<endl;
71              cout<<"2. Tambah Belakang"<<endl;
72              cout<<"3. Tampilkan Data"<<endl;
73              cout<<"4. Hapus Depan"<<endl;
74              cout<<"5. Hapus Belakang"<<endl;
75              cout<<"6. Reset"<<endl;
76              cout<<"7. Kembali ke Menu"<<endl;
77              cout<<"Pilihan : ";
78              cin>>pilihan;
79              pil=atoi(pilihan);
80
81              switch(pil) {
82              case 1:
83                  tambahDepanH();
84                  break;
85              case 2:
86                  tambahBelakangH();
87                  break;
88              case 3:
89                  tampilkanH();
90                  break;
91              case 4:
92                  hapusDepanH();
93                  break;
94              case 5:
95                  hapusBelakangH();
96                  break;
97              case 6:
98                  clearH();
99                  break;
100             default:
101                 system("clear");
102                 goto menu;
103             }
104
105             cout<<"\n<press any key to
    continue>"<<endl;
106             getch();
107             system("clear");
108
109         } while (pil!=7);
110     } else if(menu==2){
```

```
        do {
            cout<<"Double Linked List Non Circular (DLLNC) (Head dan Tail)"<<endl;

cout<<"==========================================="<<endl;
            cout<<"1. Tambah Depan"<<endl;
            cout<<"2. Tambah Belakang"<<endl;
            cout<<"3. Tampilkan Data"<<endl;
            cout<<"4. Hapus Depan"<<endl;
            cout<<"5. Hapus Belakang"<<endl;
            cout<<"6. Reset"<<endl;
            cout<<"7. Kembali ke Menu"<<endl;
            cout<<"Pilihan : ";
            cin>>pilihan;
            pil=atoi(pilihan);

            switch(pil) {
            case 1:
                tambahDepanHT();
                break;
            case 2:
                tambahBelakangHT();
                break;
            case 3:
                tampilkanHT();
                break;
            case 4:
                hapusDepanHT();
                break;
            case 5:
                hapusBelakangHT();
                break;
            case 6:
                clearHT();
                break;
            default:
                system("clear");
                goto menu;
            }

            cout<<"\npress any key to continue"<<endl;
            getch();
            system("clear");
```

```

        } while (pil<7);
    } else {
        cout<<"\nTERIMA KASIH"<<endl;
        cout<<"Program was made by Aufa Thariq Akram (2510817310014)."<<endl;
    }
}

void initH() {
    head = NULL;
}

void initHT() {
    head = NULL;
    tail = NULL;
}

int isEmptyH() {
    if(head == NULL) return 1;
    else return 0;
}

int isEmptyHT() {
    if(tail == NULL) return 1;
    else return 0;
}

void tambahDepanH() {
    cout<<"Masukkan data : ";
    cin >> dataBaru;
    TNode *baru;
    baru = new TNode;
    baru->data = dataBaru;
    baru->next = NULL;
    baru->prev = NULL;
    if (isEmptyH() == 1) {
        head = baru;
    } else {
        baru->next = head;
        head->prev = baru;
        head = baru;
    }
    cout << "Data \""<<dataBaru<<"\" berhasil dimasukkan di bagian depan.";
```

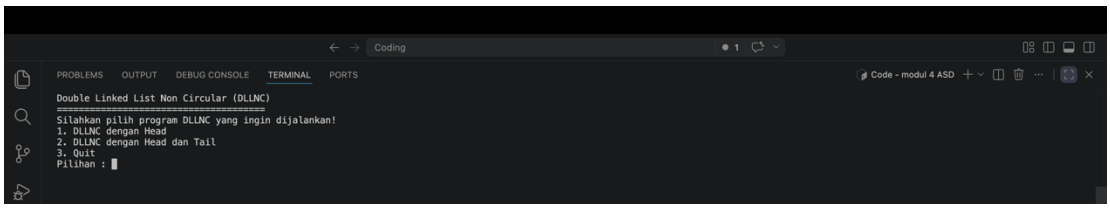
```
196 }
197
198 void tambahDepanHT() {
199     cout<<"Masukkan data : ";
200     cin>>dataBaru;
201     TNode *baru;
202     baru = new TNode;
203     baru->data = dataBaru;
204     baru->next = NULL;
205     baru->prev = NULL;
206     if(isEmptyHT() == 1) {
207         head = baru;
208         tail = baru;
209     } else {
210         baru->next = head;
211         head->prev = baru;
212         head = baru;
213     }
214     cout << "Data \""<<dataBaru<<"\" berhasil
    dimasukkan di bagian depan.";
215 }
216
217 void tambahBelakangH() {
218     cout<<"Masukkan data : ";
219     cin>>dataBaru;
220     TNode *baru, *bantu;
221     baru = new TNode;
222     baru->data = dataBaru;
223     baru->next = NULL;
224     baru->prev = NULL;
225     if(isEmptyH() == 1) {
226         head = baru;
227     } else {
228         bantu = head;
229         while(bantu->next != NULL){
230             bantu = bantu->next;
231         }
232         bantu->next = baru;
233         baru->prev = bantu;
234     }
235     cout << "Data \""<<dataBaru<<"\" berhasil
    dimasukkan di bagian belakang.";
236 }
237
238 void tambahBelakangHT() {
```

```
    cout<<"Masukkan data : ";
    cin>>dataBaru;
    TNode *baru;
    baru = new TNode;
    baru->data = dataBaru;
    baru->next = NULL;
    baru->prev = NULL;
    if(isEmptyHT() == 1) {
        head = baru;
        tail = baru;
    } else {
        tail->next = baru;
        baru->prev = tail;
        tail = baru;
    }
    cout << "Data \""<<dataBaru<<"\" berhasil dimasukkan di bagian belakang.";
}

void tampilkanH() {
    TNode *bantu;
    bantu = head;
    if(isEmptyH() == 0) {
        while(bantu != NULL) {
            cout<<bantu->data<<' ';
            bantu = bantu->next;
        }
        cout<<endl;
    } else cout<<"Tidak terdapat data pada Linked List";
}

void tampilkanHT() {
    TNode *bantu;
    bantu = head;
    if(isEmptyHT() == 0) {
        while(bantu != tail->next) {
            cout<<bantu->data<<' ';
            bantu = bantu->next;
        }
        cout<<endl;
    } else cout<<"Tidak terdapat data pada Linked List";
}

```

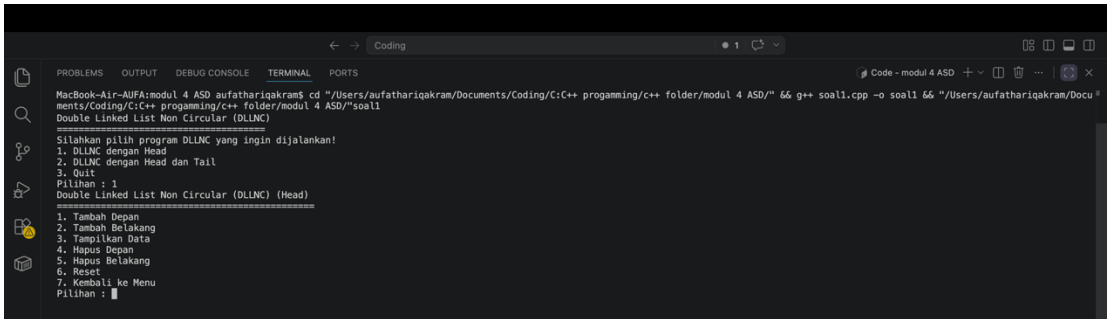
```
281 void hapusDepanH() {
282     TNode *hapus;
283     string data;
284     if(isEmptyH() == 0) {
285         hapus = head;
286         data = hapus->data;
287         if(head->next != NULL) {
288             head = head->next;
289             head->prev = NULL;
290         } else {
291             initH();
292         }
293         delete hapus;
294         cout<<"Data \""<<data<<"\" yang berada di
    depan telah berhasil dihapus.";
295     } else cout<<"Tidak terdapat data pada Linked
    List";
296 }
297
298 void hapusDepanHT() {
299     TNode *hapus;
300     string data;
301     if(isEmptyHT() == 0) {
302         hapus = head;
303         data = hapus->data;
304         if(head->next != NULL) {
305             head = head->next;
306             head->prev = NULL;
307         } else {
308             initHT();
309         }
310         delete hapus;
311         cout<<"Data \""<<data<<"\" yang berada di
    depan telah berhasil dihapus.";
312     } else cout<<"Tidak terdapat data pada Linked
    List";
313 }
314
315 void hapusBelakangH() {
316     TNode *hapus;
317     string data;
318     if(isEmptyH() == 0) {
319         hapus = head;
320         while(hapus->next != NULL){
321             hapus = hapus->next;
```

```
        }
        data = hapus->data;
        if(head->next != NULL) {
            hapus->prev->next = NULL;
        } else {
            initH();
        }
        delete hapus;
        cout<<"Data \""<<data<<"\" yang berada di belakang telah berhasil dihapus.";
    } else cout<<"Tidak terdapat data pada Linked List";
}

void hapusBelakangHT() {
    TNode *hapus;
    string data;
    if(isEmptyHT() == 0) {
        hapus = tail;
        data = hapus->data;
        if(head->next != NULL) {
            tail = tail->prev;
            tail->next = NULL;
        } else {
            initHT();
        }
        delete hapus;
        cout<<"Data \""<<data<<"\" yang berada di belakang telah berhasil dihapus.";
    } else cout<<"Tidak terdapat data pada Linked List";
}

void clearH() {
    TNode *bantu, *hapus;
    bantu = head;
    while(bantu != NULL) {
        hapus = bantu;
        bantu = bantu->next;
        delete hapus;
    }
    initH();
    cout << "Seluruh data pada Linked List telah dibersihkan.";
}
```

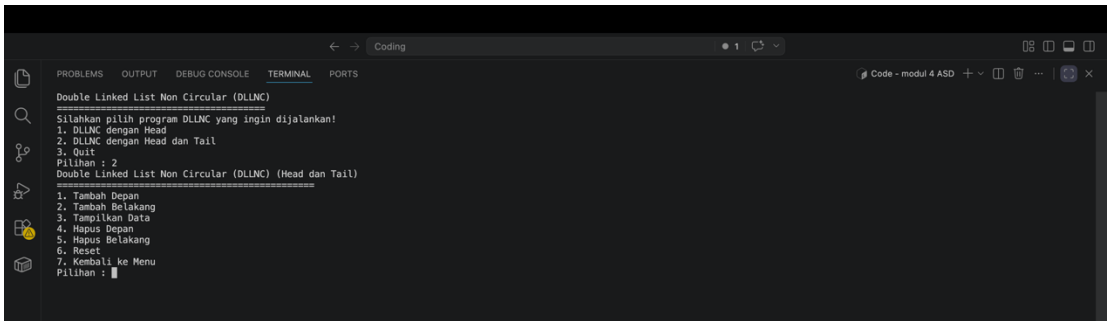
```

void clearHT() {
    TNode *bantu, *hapus;
    bantu = head;
    while(bantu != NULL) {
        hapus = bantu;
        bantu = bantu->next;
        delete hapus;
    }
    initHT();
    cout << "Seluruh data pada Linked List telah dibersihkan.";
}
```



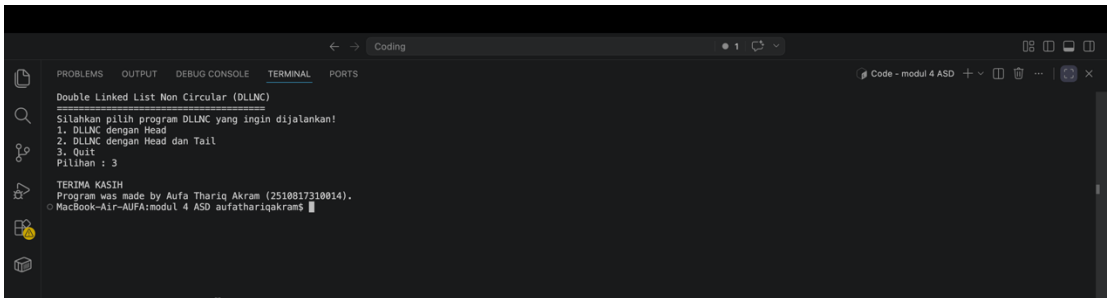
Gambar 1 Screenshot Output Program Soal 1 "Tampilan awal Program"



Gambar 2. Screenshot Output Program Soal 1 "Opsi 1"



Gambar 3 Screenshot Output Program Soal 1 "Opsi 2"



Gambar 4 Screenshot Output Program Soal 1 "Opsi 3"

- baris 1 - 4, syntax #include , #include <stdlib.h>, #include <termios.h>, dan #include <unistd.h> berfungsi untuk memanggil library atau pustaka bawaan dari bahasa C++ dan sistem operasi macOS agar program memiliki kemampuan dan instruksi dasar yang siap pakai. Library iostream digunakan untuk menjalankan proses komunikasi input dan output data standar di layar, stdlib.h digunakan untuk dapat mengeksekusi perintah manipulasi sistem seperti membersihkan layar terminal yang penuh, sedangkan termios.h dan unistd.h adalah library khusus yang ada pada ekosistem berbasis UNIX (seperti Mac) yang bertugas untuk mengatur antarmuka terminal tingkat rendah dan mendeteksi penekanan tombol secara langsung dari perangkat keras keyboard.
- baris 6, syntax using namespace std; berfungsi untuk memberikan instruksi kepada kompiler bahasa C++ agar menggunakan ruang nama standar secara otomatis untuk keseluruhan file. Hal ini sangat berguna untuk mempersingkat dan merapikan penulisan kode dasar sehingga kita tidak perlu mengetikkan awalan spesifik std:: secara berulang kali di setiap kita memanggil perintah dasar seperti perintah cetak teks ke layar (cout), perintah menerima ketikan (cin), atau perintah pembuatan baris baru (endl).
- baris 8 - 18, syntax int getch() berfungsi untuk mendefinisikan sebuah fungsi kustom atau buatan sendiri sebagai solusi pengganti dari fungsi bawaan library conio.h yang sifatnya tertutup dan hanya tersedia secara eksklusif di OS Windows. Fungsi ini bekerja dengan memanipulasi pengaturan asli dari terminal macOS melalui cara mematikan fitur echo (menampilkan karakter) dan mode kanonikal (menunggu enter) untuk sementara waktu, sehingga program dapat langsung

membaca dan menangkap input satu buah karakter bebas dari penekanan tombol keyboard pengguna dengan instan, lalu setelah karakter ditangkap ia akan langsung mengembalikan pengaturan terminal ke kondisi normal seperti semula tanpa merusak sistem.

- baris 20 - 24, syntax typedef struct TNode berfungsi untuk merancang dan menciptakan sebuah cetakan struktur tipe data kustom yang bisa diibaratkan seperti sebuah gerbong kereta, di mana di dalam satu gerbong tersebut tersimpan tiga buah komponen utama yang saling bekerja sama. Komponen pertama adalah variabel dengan tipe string bernama data yang dialokasikan khusus untuk menyimpan teks murni yang diinputkan pengguna, sedangkan dua komponen lainnya merupakan variabel berjenis pointer bernama *next untuk menyimpan arah rute atau alamat memori dari gerbong data setelahnya (untuk arah maju), dan pointer *prev untuk menyimpan arah rute gerbong data sebelumnya (untuk arah mundur), yang mana ini menjadi identitas dan ciri khas utama dari mekanisme Double Linked List.
- baris 26, syntax TNode *head, *tail; berfungsi untuk mendeklarasikan dua buah variabel pointer utama berskala global yang akan selalu beroperasi menjadi titik acuan atau rambu penanda jalan dari seluruh antrean gerbong data. Pointer dengan nama head diberikan tugas untuk selalu menempel, mengawasi, dan menunjuk pada alamat gerbong data urutan paling pertama di dalam memori, sedangkan pointer dengan nama tail diberikan tugas untuk selalu menempel dan memantau gerbong data urutan paling ujung belakang, sehingga hal ini sangat memudahkan sistem program untuk melacak di mana letak ujung awal maupun ujung akhir dari sekumpulan alokasi data yang saling tersambung satu sama lain di dalam ruang memori komputer.
- baris 28 - 30, syntax int pil, menu;, char pilihan[1];, dan string dataBaru; berfungsi untuk memesan dan menyiapkan ruang-ruang penyimpanan memori kosong yang bersifat sementara dan masing-masing memiliki tipe data berbeda yang disesuaikan dengan peruntukannya. Ruang memori ini sangat diperlukan untuk menampung input pilihan angka saat pengguna sedang berada di halaman menu

utama, menyimpan susunan karakter dari opsi menu yang tidak sengaja diketik salah, serta difungsikan sebagai keranjang penampung sementara untuk menyimpan teks dari data yang baru saja diketikkan oleh pengguna sebelum teks tersebut benar-benar dimasukkan atau dikaitkan ke dalam susunan gerbong Linked List yang sesungguhnya.

- baris 32 - 47, syntax void initH();, int isEmptyH();, dan seterusnya berfungsi untuk mendeklarasikan sekaligus mendaftarkan nama-nama fungsi, tipe pengembalian fungsi, dan parameternya ke dalam memori sistem di awal waktu sebelum isi detail kodenya benar-benar dijabarkan di bagian paling bawah program (teknik penulisan ini secara akademis disebut sebagai deklarasi prototipe). Tujuan fundamental dari langkah ini adalah untuk memberikan informasi awal kepada kompiler C++ yang membaca dari atas ke bawah bahwa fungsi-fungsi ini keberadaannya valid dan tersedia untuk digunakan, sehingga ketika program utama di atas memanggilnya sewaktu-waktu, tidak akan memunculkan pesan error atau dianggap sebagai perintah yang tidak dikenali oleh sistem.
- baris 49 - 142, syntax int main() berfungsi sebagai blok program sentral dan paling krusial yang akan dieksekusi pertama kali oleh prosesor komputer tepat ketika kode aplikasi mulai dijalankan. Di dalam wadah inilah terdapat keseluruhan alur logika utama yang menggerakkan dan membungkus antarmuka program, mulai dari proses merender dan menampilkan teks menu interaktif yang rapi ke layar monitor, menerapkan sistem perulangan menggunakan logika do while agar program tetap terus berjalan dan kembali ke menu awal sampai pengguna dengan sadar memilih tombol untuk keluar, hingga mengimplementasikan struktur percabangan logika switch case berlapis untuk menganalisa, mengeksekusi, dan memanggil fungsi-fungsi spesifik di luar blok (seperti perintah tambah data, perintah hapus data, atau perintah membersihkan riwayat) yang mana eksekusinya sangat bergantung dan disesuaikan dengan nilai angka pilihan yang baru saja dimasukkan oleh pengguna.
- baris 144 - 151, syntax void initH() dan void initHT() berfungsi secara spesifik untuk melakukan proses inisialisasi dasar atau tahapan pengaturan ulang kondisi

awal dari struktur Linked List pada saat program benar-benar baru pertama kali mulai berjalan atau sesaat setelah pengguna menekan tombol reset. Proses fundamental ini dilakukan dengan instruksi sederhana namun berdampak besar, yakni dengan cara memanipulasi dan memberikan nilai NULL atau kekosongan absolut secara paksa pada status pointer head maupun pointer tail, di mana hal ini ditujukan sebagai deklarasi dan penanda mutlak bagi sistem memori komputer bahwa saat ini belum ada satupun bongkahan gerbong data yang tercipta, terbentuk, atau terdaftar di dalam ruang memori komputer, sehingga rantai data dianggap sepenuhnya terputus dan kosong.

- baris 153 - 161, syntax int isEmptyH() dan int isEmptyHT() berfungsi sebagai fungsi pemeriksa, inspektur, atau validator keamanan untuk mengecek status terkini dan jumlah beban dari linked list, guna mengetahui apakah ia sedang dalam keadaan kosong melompong ataukah sudah menampung dan berisi sekumpulan data di dalamnya. Logika pengecekan yang diusung dalam fungsi ini sejatinya sangat ringkas, di mana program secara harfiah hanya perlu melihat dan memvalidasi apakah penunjuk arah head atau tail pada saat itu masih tetap menunjuk pada arah nilai NULL yang berarti kehampaan; jika kondisinya memang iya maka fungsi akan langsung mengembalikan nilai berupa angka 1 ke sistem yang berarti terkonfirmasi benar (true) bahwa memori memang kosong, dan sebaliknya ia akan langsung melempar angka 0 yang bermakna salah (false) jika sistem mendeteksi sudah ada gerbong data yang terdaftar menempatinya.
- baris 163 - 178, syntax void tambahDepanH() berfungsi untuk proses perakitan dan penciptaan gerbong memori data yang benar-benar baru melalui seruan perintah new TNode dan langsung bergerak menyisipkannya untuk menempati urutan paling awal pada formasi rantai data, di mana fungsi ini dikhususkan pada tipe struktur algoritma yang hanya mengandalkan satu rambu Head saja. Logika prosedur kerjanya diawali dengan menyuntikkan teks inputan ke variabel data yang baru lahir tersebut, lalu sistem akan mendeteksi ruang; jika ruang tersebut terkonfirmasi masih kosong murni maka gerbong data tunggal ini langsung dipromosikan dan dinobatkan menjadi sang head sejati, namun apabila sistem

mendeteksi sudah ada gerbong data peninggalan lain yang mengantre, maka tali pengait next pada gerbong baru ini akan lebih dulu dihubungkan ke gerbong head yang lama, kemudian tali prev pada gerbong head lama diikatkan memutar kembali untuk menyambung ke gerbong baru, sebelum pada tahap final posisi rambu kebesaran head resmi digeser dan dipindahkan ke gerbong yang paling baru saja diciptakan tersebut agar menjadi pemimpin barisan di urutan pertama.

- baris 180 - 196, syntax void tambahDepanHT() berfungsi untuk merealisasikan penambahan elemen data baru tepat di bagian ujung paling depan, hanya saja dengan melibatkan pendekatan sistem koordinasi yang sedikit diadaptasi karena tipe ini melibatkan dua buah penunjuk jalan sakral sekaligus yaitu rambu Head di depan dan rambu Tail di belakang. Secara abstraksi konsep dan kerangka logikanya mekanisme ini diklaim hampir sama persis dan identik dengan gaya kerja fungsi versi pendahulunya, yang membedakannya dengan tajam hanyalah saat sistem mendeteksi kondisi memori sedang dalam masa benar-benar bersih dan kosong murni, di mana selain elemen data yang baru lahir tersebut otomatis diangkat menjadi penguasa head yang menunjuk ke garis awal mula, ia juga secara simultan pada detik yang sama diangkat paksa menjadi pemegang kendali tail yang menunjuk ke garis penutup akhir, lantaran secara eksistensial ia adalah satu-satunya entitas gerbong data tunggal yang hidup dan menyusun antrean di seluruh jagat memori program pada saat itu.
- baris 198 - 216, syntax void tambahBelakangH() berfungsi untuk melempar, menempatkan, dan memposisikan gerbong data yang baru dibuat agar bersandar tepat di ujung posisi belakang barisan atau antrean paling ekor dari keseluruhan data yang mengular. Karena versi spesifik ini tidak dibekali atau dipersenjatai dengan rambu tail yang mampu memantau titik akhir secara instan, maka sebagai jalan keluar program terpaksa mengorbankan waktu komputasi dengan mendelegasikan sebuah pointer asisten atau pointer bantu ekstra untuk diterjunkan guna melakukan proses penelusuran manual (dalam ilmu komputer dikenal sebagai proses traversing), di mana pointer ini akan melompat satu per satu dari gerbong data pertama menyusuri rantainya berjalan merayap mundur ke belakang

memanfaatkan putaran perulangan while sampai ia akhirnya membentur gerbong data terakhir yang tali pengait next-nya menunjukkan nilai NULL yang mengindikasikan itu adalah dinding ujungnya, lalu pada detik itulah gerbong data baru seketika diikatkan dengan kokoh untuk meneruskan ekor gerbong paling belakang tersebut.

- baris 218 - 234, syntax void tambahBelakangHT() berfungsi untuk mengintegrasikan dan meletakkan elemen gerbong data baru di antrean titik paling belakang dengan strategi komputasi yang direkayasa agar jauh lebih sigap, instan, efisien, dan menghemat sumber daya sistem. Alasan logis di balik kecepatan operasi eksekusi pada fungsi versi penyempurnaan ini dikarenakan program sudah secara mandiri memiliki arsitektur yang menempatkan pointer eksekutif tail yang diberi mandat eksklusif untuk terus menetap, memantau, dan memaku posisinya selalu berjaga di titik koordinat data urutan paling penutup, sehingga saat ada sinyal transmisi elemen data baru yang hendak disusupkan masuk, program kini tidak perlu lagi bersusah payah membuang energi untuk merayapi dan menelusuri data berulang-ulang dari titik terdepan, melainkan bisa seketika langsung mengaitkan rantai elemen baru tersebut menempel presisi di garis belakang si rambu tail yang sedang berjaga di sana, lalu merubah dan memindahkan jabatan kepemilikan tail ke tubuh elemen data termuda tersebut secara estafet.
- baris 236 - 256, syntax void tampilkanH() dan void tampilkanHT() berfungsi untuk mengekstrak, membaca pelan-pelan, dan memproyeksikan keseluruhan nilai teks murni dari masing-masing rongga gerbong data yang tersusun rapi di dalam dimensi Linked List supaya informasi wujud aslinya bisa dilihat dengan jelas oleh bola mata pengguna pada layar monitor terminal yang membentang. Proses interpretasi ini dijalankan dengan mekanisme menaruh sesosok pointer kurir atau asisten bantu berpijak memposisikan dirinya tepat menimpa koordinat head paling awal, kemudian ia akan dipaksa untuk masuk merambah ke dalam terowongan perulangan while di mana tugas pertamanya adalah mencetak isi tulisan dari variabel data pada gerbong tempat ia berdiri sebelum pada langkah berikutnya sang pointer asisten tersebut akan diperintahkan bergeser melangkah maju secara

linear mengikuti petunjuk tali pengikat next menyusuri dari gerbong yang satu beranjak ke gerbong sebelahnya secara bertahap dan terstruktur layaknya orang membaca buku, di mana rutinitas estafet ini akan terus berulang secara konstan tanpa henti sampai akhirnya langkahnya dihentikan paksa saat ia berhadapan langsung dan menemui simbol nilai hampa NULL di ambang jurang ujung antrean yang berarti tidak ada data tersisa lagi untuk dicetak ke luar.

- baris 258 - 288, syntax void hapusDepanH() dan void hapusDepanHT() berfungsi untuk mendelegasikan tugas mengeksekusi operasi pemotongan rantai dan penghilangan wujud presensi elemen data yang bermukim dengan status menempati tepat di koordinat urutan paling permulaan di mana pointer head bernaung. Langkah eksekusi logikanya diurutkan sedemikian rupa demi meminimalisir risiko kehilangan jejak sisa antrean, di mana secara runtut program akan mengamankan dan mencadangkan titik posisi data pertama itu ke dalam genggaman sebuah pointer sementara khusus yang dilabeli dengan nama hapus, lalu dengan sigap program akan memajukan dan menyerahkan kekuasaan koordinat pointer head murni ke gerbong data pengantre di urutan nomor dua, dan sesudah wilayah koordinat head baru tersebut dinyatakan telah aman berpindah tempat, maka rantai tali pengikat laju mundur (prev) pada badan head yang baru didapuk tersebut akan segera dicincang dan diputus secara paksa dengan cara memberinya status injeksi nilai hampa NULL mutlak, lalu setelah proses transisi isolasi itu rampung dilaksanakan barulah eksistensi gerbong data urutan perdana malang yang sudah dikarantina dengan dilabeli pointer hapus di awal tadi pada akhirnya dimusnahkan dan dihapus secara harfiah dan permanen sepenuhnya dari lanskap susunan sistem memori komputer menggunakan ketegasan sabetan perintah delete memori dasar.
- baris 290 - 323, syntax void hapusBelakangH() dan void hapusBelakangHT() berfungsi untuk menyelenggarakan tahapan isolasi dan menghilangkan perwujudan gerbong data yang lokasinya menempati rel posisi paling merana di ujung batasan akhir penutup gerbong belakang barisan. Operasi penghapusan terbalik ini memuat dan menggendong komparasi perbedaan pondasi yang cukup

kentara di antara kedua fungsinya; bilamana pada varian head saja, sistem program diwajibkan mutlak untuk bersusah payah merangkak dan mengecek langkah maju menelusuri dari garis peluit awal mencari letak ujung bongkahan sebelum ia dapat merusak dan memutus rantai tali penarik pada gerbong pra-terakhir tersebut secara logis, sedangkan sebaliknya bila menilik pada format varian head lengkap dengan tail rupanya tata cara operasionalnya bergulir berkali lipat lebih pragmatis dan instan murni dikarenakan mekanisme program cukup membangkitkan dan menunggangi kapabilitas rambu penanda tail sakti yang notabene memang sudah setia berdiam berjaga konstan di batas jurang gerbong ujung, dan dari titik berdiri sang tail itulah program hanya menginstruksikan gerak melangkah mundur perlahan memindahkan estafet tail itu tepat satu langkah gerbong mundur saja, menyusul kemudian memutuskan jerat tali pengikat arah depan dari pijakan tail yang mundur tersebut tanpa belas kasihan, dan barulah seketika entitas data buangan yang tertinggal sendirian terisolir tanpa induk sambungan di seberang sana pada ujung paling belakang itu diamputasi dan disirnakan seutuhnya dari peta topografi memori sistem.

- baris 325 - 345, syntax void clearH() dan void clearHT() berfungsi untuk meregulasi proses pembumihangusan massal demi meruntuhkan, menghancurkan, membakar, serta meluluhlantakkan keseluruhan isi konstruksi jaring rantai elemen Linked List tersebut secara tidak bersisa dan seraya membebaskan kembali kapasitas bilik-bilik okupansi memori RAM komputer yang tadinya disita agar terbebas secara paripurna dan merata di segala titik koordinat penjuru sistem. Anatomi proses pembersihan genosida ekosistem yang terprogram masif ini beroperasi layaknya ekskavator atau armada alat penyedot debu ganas yang tidak pandang bulu, direalisasikan menggunakan intrik menempatkan sesosok pointer pembersih asisten di garda pijakan head titik pangkal awal mula, lalu memacunya beringas menelusuri deretan baris per baris ke arah zona terbelakang mendompleng sirkuit putaran perulangan berantai sembari asisten ini didikte keras untuk terus menerus menandai sekaligus menebas dan membongkar (melalui operasi native delete) per satuan setiap fisik bilik data yang diinjaki dan

dilewatinya tanpa sedikitpun mengendorkan ritme perburuan hingga sisa reruntuhannya benar-benar punah tidak tersisa serbuk memori pun, disusul kemudian seluruh kekacauan pembersihan dramatis itu dilabuhkan dan diakhiri secara seremonial dengan cara memanggil reinkarnasi kembali mandat dari fungsi init() yang senyap guna menurunkan tensi program dan menormalkan saklar reset di mana takhta status memori komputer ditahbiskan kembali suci, hampa, dan steril seratus persen murni mengisyaratkan sistem kini siap menyongsong siklus pengisian tatanan peradaban rantai gerbong baru di masa depan kelak.

SOAL 2

Apa fungsi next pada coding?

Pembahasan

Dalam teori struktur data, variabel next merupakan elemen penunjuk atau pointer yang dialokasikan khusus untuk menyimpan alamat memori dari simpul data yang berada tepat setelah simpul tersebut bernaung. Fungsinya adalah untuk menciptakan alur navigasi maju secara linear, sehingga sistem dapat mengenali urutan data berikutnya dalam sebuah rangkaian yang tersambung secara dinamis di dalam memori komputer tanpa harus menyimpan data tersebut secara berdekatan.

Pada implementasi codingan soal 1, variabel next digunakan untuk merangkai simpul-simpul data agar menjadi satu kesatuan, seperti terlihat pada instruksi baru->next = head yang mengarahkan penunjuk simpul baru ke alamat yang sebelumnya dipegang oleh head. Selain itu, fungsi next juga terlihat pada proses penelusuran data di dalam fungsi tampilkan, di mana perintah bantu = bantu->next memaksa pointer asisten untuk melompat ke alamat memori simpul selanjutnya agar seluruh isi data dapat dibaca dan ditampilkan ke layar secara berurutan.

SOAL 3

Apa fungsi prev pada coding?

Pembahasan

Variabel prev secara teoritis merupakan pointer pendamping yang bertugas menyimpan alamat memori dari simpul data yang terletak tepat sebelum simpul tersebut dalam sebuah rangkaian. Keberadaan prev memungkinkan terjadinya pergerakan data dua arah atau bersifat dua arah, yang memberikan fleksibilitas bagi program untuk melakukan pelacakan mundur kembali ke titik data sebelumnya tanpa harus memulai proses pembacaan ulang dari titik koordinat awal barisan.

Dalam konteks codingan soal 1, variabel prev berperan vital untuk mempertegas identitas sebagai Double Linked List melalui perintah seperti head->prev = baru yang bertugas mengikat kembali simpul lama ke arah simpul yang baru masuk di depannya. Hubungan timbal balik ini memastikan bahwa setiap gerbong data tidak hanya tahu siapa yang ada di depannya melalui pointer next, tetapi juga memiliki informasi akurat mengenai siapa yang berada di belakangnya melalui pointer prev, sehingga struktur data tetap kokoh dan saling terhubung dari dua sisi.

SOAL 4

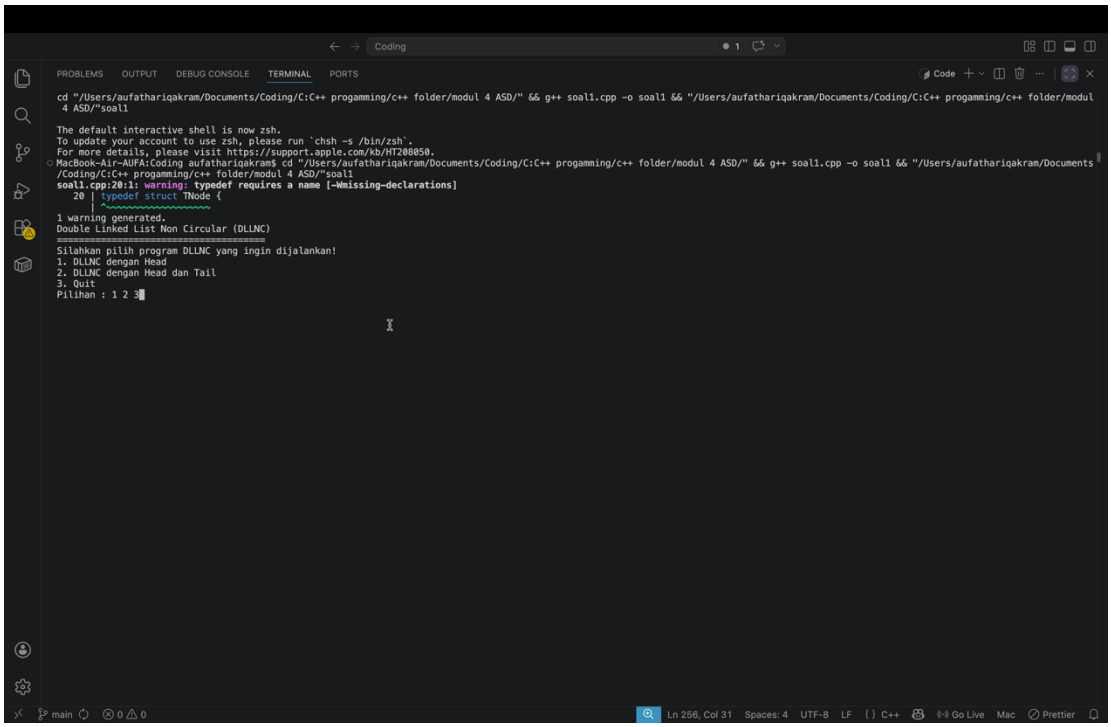
Gantilah baris 244 dan 256 dari cout<<bantu->data<<' '; menjadi cout<<head->data<<' '; lalu jawab pertanyaan berikut :

A. Apa yang terjadi jika anda menambahkan beberapa data pada program lalu tampilkan datanya, dan screenshoot hasilnya.s

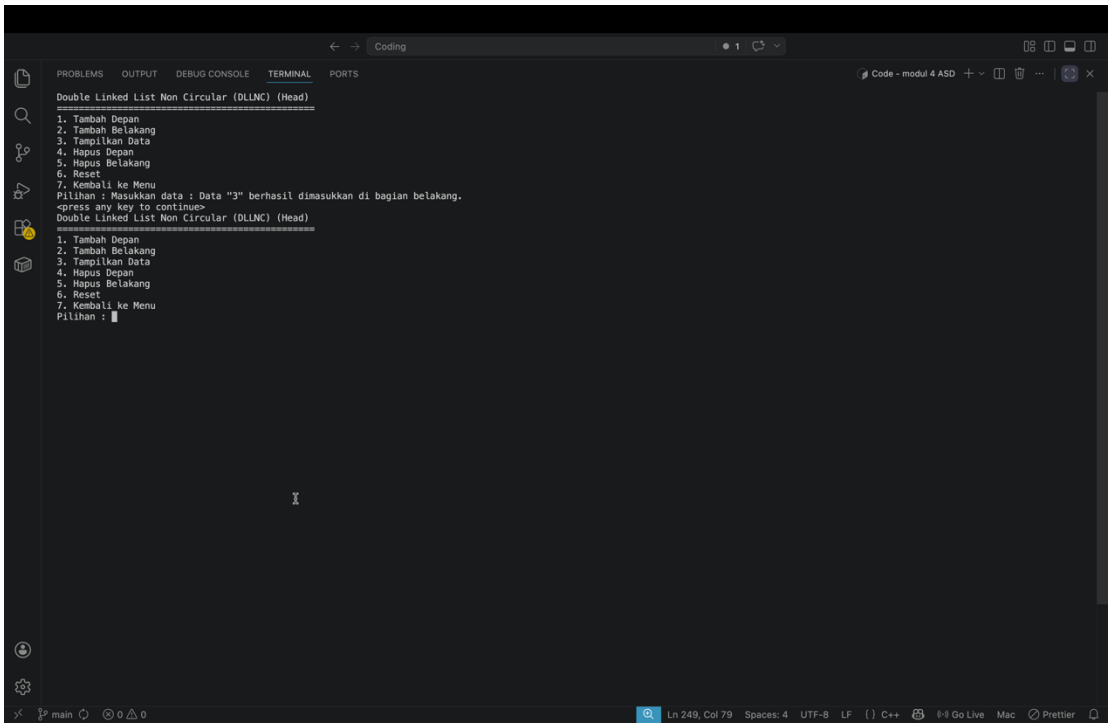
B. Jelaskan mengapa hal tersebut bisa terjadi dan data apa yang ditampilkan oleh program?

Pembahasan

A. Setelah beberapa data dimasukkan (misalnya: 1 2 3) dan menu tampilkan dipilih, program akan mencetak data yang pertama kali dimasukkan secara berulang-ulang sebanyak jumlah total data yang ada.



Gambar 5 Screenshot Output Program Soal 4 "Input 1 2 3"



Gambar 6 Screenshot Output Program Soal 4 "Hasil input 1 2 3"

B. Hal ini terjadi karena adanya kesalahan logika pada instruksi output. Di dalam perulangan while, variabel bantu sebenarnya sudah bergerak maju ke simpul berikutnya melalui perintah bantu = bantu->next. Namun, karena perintah cetak (cout) secara statis diubah untuk memanggil head->data, program akan selalu mengambil nilai dari simpul pertama (head) di setiap putaran perulangan, sehingga data lain di dalam list tidak pernah ditampilkan.

LINK GITHUB

https://github.com/IntraGH/Algoritma-dan-Struktur-Data/tree/main/Modul%204